



Universidade Federal de Goiás
Instituto de Matemática e Estatística
Bacharelado em Matemática Aplicada e Computacional

Componente Curricular: INF0286 - ALGORITMOS E ESTRUTURAS DE DADOS 1
Turma A: 2025/2
Professora: Dra. Renata Dutra Braga

Lista 05 (23/09/25)

1) Implemente uma pilha de inteiros com vetor de tamanho fixo (MAX). Crie as funções `init`, `empty`, `full`, `push`, `pop`. No `main`, empilhe 5 notas (ex.: 7, 8, 9, 6, 10) e depois desempilhe tudo, mostrando na tela para evidenciar a ordem LIFO.

Sugestões de técnicas (C) e estrutura.

- `typedef struct` com `int v[MAX]` e `int topo`.
- Checagem de limites (under/overflow).
- Bibliotecas: `<stdio.h>`, `<stdbool.h>`.
- Estrutura: pilha estática (vetor fixo).

2) Reaproveite a pilha do exercício 1 e acrescente a função `top` que consulta o elemento do topo sem removê-lo. No `main`, empilhe três valores e mostre o topo, sem alterar a pilha.

Sugestões de técnicas (C) e estrutura.

- Mesmo `struct` do ex. 1; função `top(Stack*, int*)`.
- Estrutura: pilha estática.

3) Leia uma palavra (sem espaços) e imprima-a invertida usando uma pilha de `char`. O código deve empilhar cada caractere e depois desempilhar para formar a inversão.

Sugestões de técnicas (C) e estrutura.

- `char v[MAX]` na pilha; leitura com `scanf("%s", ...)`.
- Estrutura: pilha estática de `char`.

4) Simule a sequência de chamadas: `main` -> `f1` -> `f2`. Empilhe os nomes das funções conforme “entram” e desempilhe ao “retornar”, imprimindo mensagens que mostrem a ordem LIFO.

Sugestões de técnicas (C) e estrutura.

- Vetor de `char [SIZE] [LEN]` para armazenar nomes.
- Estrutura: pilha estática de strings.

5) Implemente uma fila circular de nomes (`char [LEN]`). Funções: `init`, `empty`, `full`, `enqueue`, `dequeue`, `front`. Enfileire “Ana”, “Beto”, “Clara” e atenda na ordem (FIFO).

Sugestões de técnicas (C) e estrutura.

- Índices `in`, `out` e contador `cnt`.
- Operador módulo `%N`.
- Estrutura: fila circular estática de strings

6) Com a mesma fila do exercício 5, enfileire três nomes e utilize `front` para consultar quem é o próximo a ser atendido, sem removê-lo. Mostre o nome e depois atenda todos.

Sugestões de técnicas (C) e estrutura.

- Reutilize `front` + `dequeue`.
- Estrutura: fila circular estática

7) Simule uma fila de impressão: enfileire 4 nomes de arquivos (“a.pdf”, “b.doc”, ...) e desenfileire todos, imprimindo a ordem de processamento.

Sugestões de técnicas (C) e estrutura.

- Mesmo TAD do ex. 5; apenas nomes diferentes.
- Estrutura: fila circular estática

8) Coloque 5 alunos em uma fila. Atenda 10 passos: a cada passo, pegue o primeiro, imprima “Atendido: X” e reenfileire o mesmo nome no final (simulando rodízio). Mostre a sequência produzida.

Sugestões de técnicas (C) e estrutura.

- Reutilize `dequeue` seguido de `enqueue`.

- Estrutura: fila circular estática.

9) Implemente uma lista sequencial (vetor fixo) de `int` com `size` atual. Funções: `push_back`, `print`, `remove_first(value)` (remove a primeira ocorrência, deslocando elementos). Demonstre inserindo 1,2,3 e removendo 2.

Sugestões de técnicas (C) e estrutura.

- Vetor `int a[MAX] + int size`.
- Deslocamento com laço `for`.
- Estrutura: lista sequencial estática.

10) Crie uma lista simplesmente encadeada de `int` com `insert_front` e `print`. Insira na ordem: 5, depois 7, depois 9 (sempre no início) e mostre a lista resultante.

Sugestões de técnicas (C) e estrutura.

- `typedef struct Node { int v; struct Node* next; }.`
- `malloc/free` quando necessário.
- Estrutura: lista encadeada simples.

11) Na lista do exercício 10, implemente `find(Node* h, int x)` que retorna ponteiro para o nó com valor `x` (ou `NULL` se não existir). Use para buscar o valor 7 e imprima uma mensagem adequada.

Sugestões de técnicas (C) e estrutura.

- Percurso com `for/while`.
- Estrutura: lista encadeada simples.

12) Implemente `remove_first(Node** h, int x)` que remove a primeira ocorrência de `x` da lista (ajustando ponteiros corretamente). Teste removendo 7 da lista criada anteriormente.

Sugestões de técnicas (C) e estrutura.

- Manter ponteiros `prev` e `curr`.
- Tratar o caso de remoção na cabeça.
- Estrutura: lista encadeada simples.